



**BANGKOK
UNIVERSITY**

THE CREATIVE UNIVERSITY

หัวข้อรายงานระบบจัดการร้านตัดเสื้อ

Clothy : Tailor Shop Management System

รายวิชา : CE323 : Database Management Systems

ผู้จัดทำ

นายธนพล

ศรีทอง

1650903550

เสนอ

อ.สุกิต คุชัยสิทธิ์ (A.Sukit Kuchaisit)

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CE323 : Database Management Systems
หลักสูตรวิศวกรรมศาสตรบัณฑิต สาขาวิชาวิศวกรรมคอมพิวเตอร์และหุ่นยนต์ สำนักวิชา
วิศวกรรมศาสตร์มหาวิทยาลัยกรุงเทพ ประจำปีการศึกษาที่ 2 ปีการศึกษา 2568

1. ภาพรวมของระบบงาน ข้อกำหนดและขอบเขตการดำเนินงานของระบบ

ระบบจัดการร้านตัดเสื้อ Clothly เป็นระบบสารสนเทศสำหรับร้านตัดเสื้อขนาดเล็กถึงขนาดกลาง ออกแบบมาเพื่อช่วยให้ช่างตัดเสื้อสามารถบริหารจัดการลูกค้า ชูตงาน สัดส่วนร่างกาย และรูปภาพประกอบได้อย่างมีประสิทธิภาพ ผ่านระบบเว็บแอปพลิเคชัน

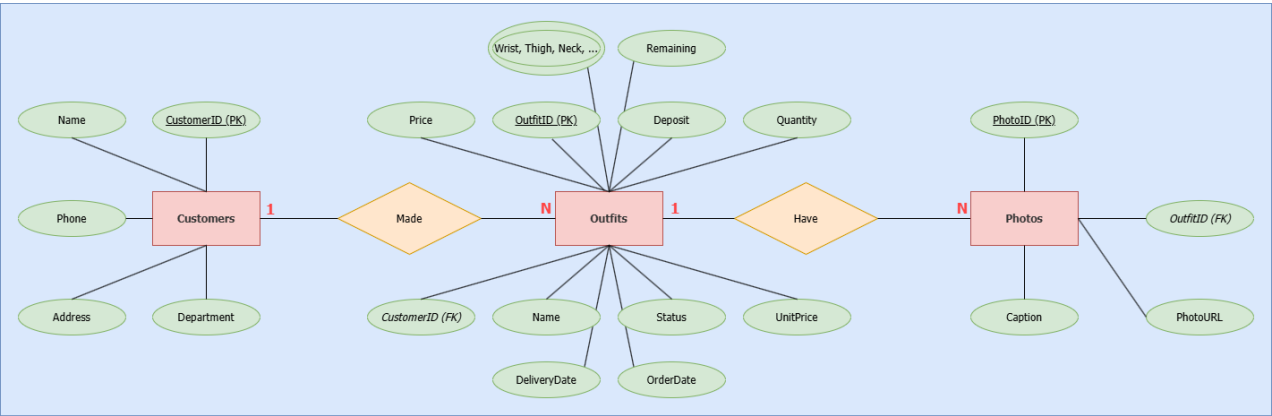
ขอบเขตการดำเนินงาน

- จัดเก็บข้อมูลลูกค้า (ชื่อ เบอร์โทรศัพท์ แผนก ที่อยู่)
- บันทึกชูตงานแต่ละชุดพร้อมสัดส่วนร่างกายทั้งหมด 24 จุด
- ติดตามสถานะงาน (รอดำเนินการ / กำลังทำ / เสร็จสิ้น)
- บันทึกข้อมูลทางการเงิน (ราคา มัดจำ ค้างชำระ)
- จัดการรูปภาพประกอบแต่ละชุด
- แสดงภาพรวมงานผ่าน Work Board (Kanban)

เทคโนโลยีที่ใช้

ส่วน	เทคโนโลยี
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	Node.js, Express.js
Database	Microsoft SQL Server (MSSQL)
ORM/Driver	mssql (npm package)

2. ER Diagram



ความสัมพันธ์ในระบบ

Customers ↔ Outfits : หนึ่งลูกค้า (1) มีได้หลายชุด (N) — ความสัมพันธ์ One-to-Many

Outfits ↔ Photos : หนึ่งชุด (1) มีได้หลายรูปภาพ (N) — ความสัมพันธ์ One-to-Many

Relation Name: Customers–Outfits = "Made", Outfits–Photos = "Have"

3. ตาราง (Map Table จาก ER Diagram)

หมายเหตุ: ชิดเส้นใต้ = Primary Key (PK), ตัวเอียง + ชิดเส้นใต้แบบเส้นปะ = Foreign Key (FK)

Customers

<u>CustomerID</u>	Name	Phone	Department	Address
-------------------	------	-------	------------	---------

Outfits

<u>OutfitID</u>	<i><u>CustomerID</u></i>	Name	Status	UnitPrice	Quantity
-----------------	--------------------------	------	--------	-----------	----------

Price	Deposit	Remaining	OrderDate	DeliveryDate	Wrist, Thigh, Crotch ...
-------	---------	-----------	-----------	--------------	-----------------------------

Photos

<u>PhotoID</u>	<i><u>OutfitID</u></i>	PhotoURL	Caption
----------------	------------------------	----------	---------

4. พจนานุกรมข้อมูล (Data Dictionary)

ตาราง: Customers

Column	Data Type	Null	Key	คำอธิบาย
CustomerID	INT	NOT NULL	PK	รหัสลูกค้า (Auto Increment)
Name	NVARCHAR(100)	NOT NULL	-	ชื่อ-นามสกุลลูกค้า
Phone	NVARCHAR(20)	NULL	-	เบอร์โทรศัพท์
Department	NVARCHAR(100)	NULL	-	แผนก/หน่วยงาน
Address	NVARCHAR(Max)	NULL	-	สถานที่จัดส่ง

ตาราง: Outfits

Column	Data Type	Null	Key	คำอธิบาย
OutfitID	INT	NOT NULL	PK	รหัสชุด (Auto Increment)
CustomerID	INT	NOT NULL	FK → Customers	รหัสลูกค้าที่สั่งซื้อ
Name	NVARCHAR(100)	NOT NULL	-	ชื่อชุด
Status	NVARCHAR(50)	NULL	-	สถานะงาน
UnitPrice	DECIMAL(10,2)	NULL	-	ราคาต่อหน่วย
Quantity	INT	NULL	-	จำนวน
Price	DECIMAL(10,2)	NULL	-	ราคารวม
Deposit	DECIMAL(10,2)	NULL	-	มัดจำ
Remaining	DECIMAL(10,2)	NULL	-	ค้างชำระ

OrderDate	DATE	NULL	-	วันที่สั่ง
DeliveryDate	DATE	NULL	-	วันที่ส่งมอบ
FrontLength	DECIMAL(5,1)	NULL	-	ความยาวหน้า (ซม.)
BackLength	DECIMAL(5,1)	NULL	-	ความยาวหลัง (ซม.)
Shoulder	DECIMAL(5,1)	NULL	-	ไหล่ (ซม.)
ChestWidth	DECIMAL(5,1)	NULL	-	อก (ซม.)
WaistWidth	DECIMAL(5,1)	NULL	-	เอว (ซม.)
HipWidth	DECIMAL(5,1)	NULL	-	สะโพก (ซม.)
UpperArmWidth	DECIMAL(5,1)	NULL	-	ต้นแขน (ซม.)
ArmLength	DECIMAL(5,1)	NULL	-	ความยาวแขน (ซม.)
SleeveWidth	DECIMAL(5,1)	NULL	-	ปลายแขน (ซม.)
WristWidth	DECIMAL(5,1)	NULL	-	ข้อมือ (ซม.)
NeckCircumference	DECIMAL(5,1)	NULL	-	รอบคอ (ซม.)
WaistCircumference	DECIMAL(5,1)	NULL	-	รอบเอว (ซม.)
HipCircumference	DECIMAL(5,1)	NULL	-	รอบสะโพก (ซม.)
ThighWidth	DECIMAL(5,1)	NULL	-	ต้นขา (ซม.)
KneeWidth	DECIMAL(5,1)	NULL	-	เข่า (ซม.)
CalfWidth	DECIMAL(5,1)	NULL	-	น่อง (ซม.)
AnkleWidth	DECIMAL(5,1)	NULL	-	ข้อเท้า (ซม.)
TrouserLength	DECIMAL(5,1)	NULL	-	ความยาวกางเกง (ซม.)
CrotchLength	DECIMAL(5,1)	NULL	-	เป้า (ซม.)
RiseLength	DECIMAL(5,1)	NULL	-	ระยะยีน (ซม.)
LegOpening	DECIMAL(5,1)	NULL	-	ปลายขา (ซม.)

Notes	NVARCHAR(MAX)	NULL	-	หมายเหตุ
-------	---------------	------	---	----------

ตาราง: Photos

Column	Data Type	Null	Key	คำอธิบาย
PhotoID	INT	NOT NULL	PK	รหัสรูปภาพ (Auto Increment)
OutfitID	INT	NOT NULL	FK → Outfits	รหัสชุดที่เกี่ยวข้อง
PhotoURL	NVARCHAR(500)	NOT NULL	-	URL หรือ path ของรูปภาพ
Caption	NVARCHAR(Max)	NULL	-	คำบรรยายรูปภาพ

5. SQL Commands รายบุคคล

คนที่ 1 เลขที่ 5 - นายธนพล ศรีทอง รหัส 1650903550

จะแบ่งเป็น 2 แบบ คือ

แบบที่ 1 รับค่าจากเว็บ (Node.js + Express)

1. Connect

```
const sql = require('mssql/msnodesqlv8');
const dbConfig = {
  server: process.env.DB_SERVER ? `${process.env.DB_SERVER}\\SQLEXPRESS` : 'localhost\\SQLEXPRESS',
  database: process.env.DB_DATABASE || 'Clothy_DataBase',
  driver: 'ODBC Driver 17 for SQL Server',
  options: { trustedConnection: true, trustServerCertificate: true },
};
async function getPool() {
  try {
    return await sql.connect(dbConfig);
  } catch (err) {
    throw new DatabaseException(err.message);
  }
}
```

```
}  
}
```

นำเข้า library mssql/msnodesqlv8 สำหรับเชื่อมต่อ SQL Server กำหนดค่าการเชื่อมต่อใน dbConfig โดยอ่านค่า server และ database จาก environment variable (.env) ถ้าไม่มีจะใช้ค่า default คือ localhost\SQLEXPRESS และ Clothly_DataBase ใช้ trustedConnection: true เพื่อเชื่อมต่อแบบ Windows Authentication โดยไม่ต้องใส่ username/password ฟังก์ชัน getPool() ทำหน้าที่เปิดการเชื่อมต่อและคืนค่า connection pool กลับมาใช้งาน

2. Select

```
app.get('/api/customers', async (req, res, next) => {  
  try {  
    const customers = await customerRepo.findAll();  
    res.json(customers.map(c => ({  
      CustomerID: c.getId(), Name: c.getName(),  
      Phone: c.getPhone(), Department: c.getDepartment(),  
      Address: c.getAddress(),  
    })));  
  } catch (err) { next(err); }  
});  
  
// query ใน CustomerRepository  
const result = await pool.request().query('SELECT * FROM Customers');
```

เมื่อเว็บส่ง GET request มาที่ /api/customers ระบบจะเรียก customerRepo.findAll() ซึ่งรัน SELECT * FROM Customers เพื่อดึงข้อมูลลูกค้าทั้งหมดออกจากรฐานข้อมูล จากนั้น map ผลลัพธ์ให้อยู่ในรูป Object แล้วส่งกลับไปในรูปแบบ JSON

3. Insert

```
app.post('/api/customers', async (req, res, next) => {  
  try {  
    const { name, phone, dept, address } = req.body;  
    const customer = await customerRepo.create(name, phone, dept, address);  
    res.status(201).json({ message: 'เพิ่มลูกค้าสำเร็จ!', customerId: customer.getId() });  
  } catch (err) { next(err); }  
});
```

```
// query ใน CustomerRepository
const id = 'c' + Date.now();
await pool.request()
  .input('CustomerID', sql.VarChar(20), id)
  .input('Name', sql.NVarChar(100), name)
  .input('Phone', sql.VarChar(15), phone)
  .input('Department', sql.NVarChar(100), dept || null)
  .input('Address', sql.NVarChar(sql.MAX), address || null)
  .query('INSERT INTO Customers (CustomerID,Name,Phone,Department,Address) VALUES
(@CustomerID,@Name,@Phone,@Department,@Address)');
```

รับข้อมูลลูกค้า (ชื่อ เบอร์โทร แผนก ที่อยู่) จากเว็บผ่าน req.body สร้าง ID อัตโนมัติโดยใช้ 'c' + Date.now() เพื่อให้ไม่ซ้ำกัน จากนั้นส่งค่าแบบ Parameterized Query ผ่าน .input() เพื่อป้องกัน SQL Injection ก่อน INSERT เข้าตาราง Customers ถ้า Department หรือ Address ไม่ได้ส่งมาจะใส่ null แทน

4. Update

```
app.put('/api/customers/:id/rename', async (req, res, next) => {
  try {
    await customerRepo.rename(req.params.id, req.body.newName);
    res.json({ message: 'แก้ไขชื่อลูกค้าสำเร็จ! });
  } catch (err) { next(err); }
});

// query ใน CustomerRepository
await pool.request()
  .input('CustomerID', sql.VarChar(20), id)
  .input('Name', sql.NVarChar(100), newName)
  .query('UPDATE Customers SET Name = @Name WHERE CustomerID = @CustomerID');
```

รับ id ของลูกค้าจาก URL parameter และรับชื่อใหม่จาก req.body.newName แล้วรัน UPDATE เพื่อแก้ไขชื่อลูกค้าในตาราง Customers ระบุเงื่อนไข WHERE CustomerID = @CustomerID เพื่อให้แก้ไขเฉพาะรายการที่ต้องการเท่านั้น

5. Delete

```
app.delete('/api/customers/:id', async (req, res, next) => {
  try {
    await customerRepo.delete(req.params.id);
```



```

    res.json({ message: 'ลบลูกค้าสำเร็จ!' });
  } catch (err) { next(err); }
});

// query ใน CustomerRepository
await pool.request()
  .input('CustomerID', sql.VarChar(20), id)
  .query('DELETE FROM Customers WHERE CustomerID = @CustomerID');

```

รับ id ของลูกค้าจาก URL parameter แล้วส่งให้ customerRepo.delete() รัน DELETE เพื่อลบข้อมูลออกจากตาราง Customers ระบุเงื่อนไข WHERE CustomerID = @CustomerID เพื่อให้ลบเฉพาะรายการที่ระบุเท่านั้น ถ้าไม่พบ ID จะ throw ResourceNotFoundException

แบบที่ 2 — รันโดยตรงบน SSMS (Pure SQL)

1. Connect

```

-- เลือกใช้งาน Database
USE Clothly_Database;
GO

```

เปิด SSMS แล้วเลือก Server: localhost\SQLEXPRESS, Authentication: Windows Authentication จากนั้นรันคำสั่ง USE เพื่อเลือกฐานข้อมูล Clothly_Database ที่จะใช้งาน

2. Select

```
SELECT * FROM Customers;
```

ดึงข้อมูลทุก column และทุก row ในตาราง Customers ออกมาแสดงผล ใช้สำหรับตรวจสอบข้อมูลที่มีอยู่ในระบบ

ผลลัพธ์

Results Messages					
	CustomerID	Name	Phone	Department	Address
1	c1777881293258	ปณณ	0809300752	NULL	NULL

3.Insert

```

INSERT INTO Customers (CustomerID, Name, Phone, Department, Address)
VALUES ('c001', 'นายทองดี ไม่มีฝาก', '0812345678', 'IT', '123 ถ.พหลโยธิน กรุงเทพฯ');

```

เพิ่มข้อมูลลูกค้าใหม่ 1 รายการเข้าตาราง Customers โดยระบุค่าของแต่ละ column ให้ตรงกับลำดับที่กำหนดไว้

ผลลัพธ์

	CustomerID	Name	Phone	Department	Address
1	c001	นายทองดี ไม่ลืมฝาก	0812345678	IT	123 ถ.พหลโยธิน กรุงเทพฯ
2	c1777881293258	ปอนด์	0809300752	NULL	NULL

4.Update

```
UPDATE Customers
SET Name = 'ทองดี มีเก็บ',
    Department = 'Dev'
WHERE CustomerID = 'c001';
```

แก้ไขชื่อของลูกค้าที่มี CustomerID = 'c001' โดยใช้ WHERE เพื่อกำหนดเงื่อนไข ถ้าไม่ใส่ WHERE จะแก้ไขข้อมูลทุก row ในตาราง

ผลลัพธ์

	CustomerID	Name	Phone	Department	Address
1	c001	ทองดี มีเก็บ	0812345678	Dev	123 ถ.พหลโยธิน กรุงเทพฯ
2	c1777881293258	ปอนด์	0809300752	NULL	NULL

5.Delete

```
DELETE FROM Customers
WHERE CustomerID = 'c001';
```

ลบข้อมูลลูกค้าที่มี CustomerID = 'c001' ออกจากตาราง ต้องระบุ WHERE ทุกครั้ง มิฉะนั้นจะลบข้อมูลทั้งหมดในตาราง

ผลลัพธ์

	CustomerID	Name	Phone	Department	Address
1	c1777881293258	ปอนด์	0809300752	NULL	NULL